

Portal AgendeAi - Documentacao da API REST

Especificacao Completa de Rotas, Payloads e Regras de Negocio

Versao 1.2.0 • Data de Emissao: 18/08/2026 • Base URL: /api

1. Visao Geral e Arquitetura

Padroes de comunicacao, autenticao e formato de dados

A API do Portal AgendaAi é um serviço RESTful moderno desenvolvido em Express e TypeScript, conectado ao banco de dados Neon PostgreSQL em nuvem. A API disponibiliza endpoints para gestão de múltiplos estabelecimentos (tenants), profissionais, catálogo de serviços, turnos semanais, bloqueios pontuais e cálculo de slots de agendamento em tempo real.

Autenticacao: Rotas protegidas utilizam cabecalho "Authorization: Bearer <token_jwt>" com chave assinada e expiracao de 30 dias.

Codigos HTTP: 200 (Sucesso), 201 (Criado), 400 (Requisicao Invalida), 401 (Nao Autorizado), 404 (Nao Encontrado), 409 (Conflito de Horario/Duplicacao), 500 (Erro Interno).

2. Modulo de Estabelecimentos (Tenants)

Cadastro e consulta de empresas, temas visuais e links customizados

GET

/api/tenants

Publico

Retorna a lista de todos os saloes, clinicas e negocios parceiros cadastrados na plataforma com seus respectivos logos e cores de marca.

Parametros: Nenhum

Exemplo de Resposta (Status 200/201):

```
[
  {
    "id": "tenant-1",
    "name": "Barbearia Imperial",
    "slug": "barbearia-imperial",
    "logoUrl": "[Logo: Tesoura]",
    "description": "Cortes classicos e modernos",
    "themeColor": "indigo",
    "accentColor": "bg-indigo-600 text-white"
  }
]
```

GET

/api/tenants/:slug

Publico

Busca os dados de um estabelecimento a partir do seu slug (subcaminho da URL).

Parametros: :slug (String - obrigatorio) - Exemplo: "barbearia-imperial"

Exemplo de Resposta (Status 200/201):

```
{
  "id": "tenant-1",
  "name": "Barbearia Imperial",
  "slug": "barbearia-imperial",
  "logoUrl": "[Logo: Tesoura]",
  "description": "Cortes classicos e modernos",
  "themeColor": "indigo",
  "accentColor": "bg-indigo-600 text-white"
}
```

POST

/api/tenants

Publico

Cadastra um novo estabelecimento, seu primeiro profissional (prestador) como Administrador e a conta de usuario com senha criptografada via bcrypt. Retorna o token JWT para autenticao imediata.

Parametros: Nenhum

Payload / Request Body (JSON):

```
{
  "name": "Academia FitLife",
  "slug": "fitlife",
  "description": "Treinamento personalizado e musculacao",
  "logoUrl": "[Logo: Haltere]",
  "themeColor": "amber",
  "ownerName": "Carlos Silva",
  "email": "carlos@fitlife.com",
  "password": "senhaSegura123"
}
```

Exemplo de Resposta (Status 200/201):

```
{
  "tenant": { "id": "tenant-1723...", "name": "Academia FitLife", "slug": "fitlife", "themeColor": "amber" },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": { "id": "user-1723...", "email": "carlos@fitlife.com", "role": "provider", "providerId": "provider-1723..." }
}
```

3. Modulo de Categorias e Prestadores

Especialidades e equipe profissional

GET

/api/categories

PUBLICO

Retorna todas as categorias de atendimento cadastradas na plataforma com seus respectivos icones representativos.

Parametros: Nenhum

Exemplo de Resposta (Status 200/201):

```
[
  { "id": "cat-1", "name": "Barbearia e Cabelo", "imageUrl": "[Icone: Barbearia]" },
  { "id": "cat-2", "name": "Estetica e Cuidados", "imageUrl": "[Icone: Estetica]" },
  { "id": "cat-3", "name": "Manicure e Pedicure", "imageUrl": "[Icone: Manicure]" }
]
```

GET

/api/providers

PUBLICO

Lista todos os profissionais prestadores de servicos, opcionalmente filtrados por estabelecimento (tenantId).

Parametros: ?tenantId (String - opcional) - Exemplo: "tenant-1"

Exemplo de Resposta (Status 200/201):

```
[
  {
    "id": "provider-1",
    "tenantId": "tenant-1",
    "categoryId": "cat-1",
    "name": "Carlos Barber",
    "email": "carlos@imperial.com",
    "bio": "Especialista em cortes classicos",
    "categoryName": "Barbearia e Cabelo",
    "categoryImage": "[Icone: Barbearia]"
  }
]
```

4. Modulo de Catalogo de Servicos

Gerenciamento de procedimentos, duracoes, precos e buffers de preparacao

GET

/api/services

PUBLICO

Consulta a lista de servicos oferecidos. Pode ser filtrado por um profissional especifico.

Parametros: ?providerId (String - opcional)

Exemplo de Resposta (Status 200/201):

```
[
  {
    "id": "service-1",
    "providerId": "provider-1",
    "name": "Corte de Cabelo Classico",
    "description": "Corte com lavagem e alinhamento",
    "durationMinutes": 30,
    "bufferMinutes": 5,
    "price": 45.00
  }
]
```

POST

/api/services

PUBLICO

Cadastra um novo servico para um profissional especifico informando duracao e buffer.

Parametros: Nenhum

Payload / Request Body (JSON):

```
{
  "providerId": "provider-1",
  "name": "Barba Terapeutica",
  "description": "Toalha quente e navalha",
  "durationMinutes": 25,
  "bufferMinutes": 5,
  "price": 35.00
}
```

Exemplo de Resposta (Status 200/201):

```
{
  "id": "service-1723...",
  "providerId": "provider-1",
  "name": "Barba Terapeutica",
  "durationMinutes": 25,
  "bufferMinutes": 5,
  "price": 35.00
}
```

PUT /api/services/:id

Publico

Edita as propriedades de um servico existente.

Parametros: :id (String - obrigatorio) - ID do servico

Payload / Request Body (JSON):

```
{
  "name": "Barba Terapeutica Premium",
  "description": "Inclui hidratacao facial.",
  "durationMinutes": 30,
  "bufferMinutes": 5,
  "price": 40.00
}
```

Exemplo de Resposta (Status 200/201):

```
{
  "id": "service-1",
  "name": "Barba Terapeutica Premium",
  "price": 40.00
}
```

DELETE /api/services/:id

Publico

Exclui um servico do catalogo do profissional.

Parametros: :id (String - obrigatorio)

Exemplo de Resposta (Status 200/201):

```
{ "success": true }
```

5. Modulo de Disponibilidade Semanal

Definicao de expediente padrao e turnos de trabalho

GET /api/availability-rules

Publico

Retorna as regras de horarios de funcionamento por dia da semana (0=Domingo, 1=Segunda, ..., 6=Sabado).

Parametros: ?providerId (String - obrigatorio)

Exemplo de Resposta (Status 200/201):

```
[
  { "id": "rule-1", "providerId": "provider-1", "dayOfWeek": 1, "startTime": "09:00", "endTime": "18:00" },
  { "id": "rule-2", "providerId": "provider-1", "dayOfWeek": 2, "startTime": "09:00", "endTime": "18:00" }
]
```

PUT /api/availability-rules

Publico

Substitui em lote toda a grade semanal de horarios de atendimento do profissional.

Parametros: Nenhum

Payload / Request Body (JSON):

```
{
  "providerId": "provider-1",
  "rules": [
    { "dayOfWeek": 1, "startTime": "08:30", "endTime": "18:00" },
    { "dayOfWeek": 2, "startTime": "08:30", "endTime": "18:00" }
  ]
}
```

Exemplo de Resposta (Status 200/201):

```
[
  { "id": "rule-provider-1-...", "dayOfWeek": 1, "startTime": "08:30", "endTime": "18:00" }
]
```

6. Modulo de Bloqueios e Feriados (Exceptions)

Gerenciamento de folgas, feriados ou turnos especiais em datas especificas

GET /api/exceptions

Publico

Lista bloqueios de dias inteiros ou turnos excepcionais configurados para datas especificas.

Parametros: ?providerId (String - obrigatorio)

Exemplo de Resposta (Status 200/201):

```
[
  { "id": "exp-1", "providerId": "provider-1", "date": "2026-12-25", "isBlocked": true, "startTime": null, "endTime": null }
]
```

POST

/api/exceptions

Publico

Registra um bloqueio integral ou altera a jornada para um dia especifico.

Parametros: Nenhum

Payload / Request Body (JSON):

```
{
  "providerId": "provider-1",
  "date": "2026-12-25",
  "isBlocked": true,
  "startTime": null,
  "endTime": null
}
```

Exemplo de Resposta (Status 200/201):

```
{
  "id": "exp-provider-1-...",
  "providerId": "provider-1",
  "date": "2026-12-25",
  "isBlocked": true
}
```

DELETE

/api/exceptions/:id

Publico

Remove uma execucao ou desbloqueia a data previamente bloqueada.

Parametros: :id (String - obrigatorio)

Exemplo de Resposta (Status 200/201):

```
{ "success": true }
```

7. Motor de Slots e Agendamentos

Algoritmo de horarios livres, prevencao de conflito e reservas

GET

/api/slots

Publico

Algoritmo que calcula os horarios disponiveis em tempo real: verifica regras semanais, desconta execucoes/bloqueios, desconta agendamentos ja confirmados e soma buffers de intervalo.

Parametros: ?providerId (String - obrigatorio) & serviceId (String - obrigatorio) & date (YYYY-MM-DD - obrigatorio)

Exemplo de Resposta (Status 200/201):

```
[
  {
    "startTime": "2026-08-20T09:00:00.000Z",
    "endTime": "2026-08-20T09:30:00.000Z",
    "formattedTime": "09:00",
    "available": true
  }
]
```

GET

/api/bookings

Publico

Lista todos os agendamentos registrados para o prestador.

Parametros: ?providerId (String - obrigatorio)

Exemplo de Resposta (Status 200/201):

```
[
  {
    "id": "bk-1",
    "serviceName": "Corte de Cabelo Classico",
    "clientName": "Ana Paula",
    "clientEmail": "ana@email.com",
    "startTime": "2026-08-20T10:00:00.000Z",
    "endTime": "2026-08-20T10:30:00.000Z",
    "status": "confirmed",
    "price": 45.00
  }
]
```

POST

/api/bookings

Publico

Cria um novo agendamento com validacao atomica no PostgreSQL para impedir conflitos de horarios sobrepostos (Double Booking).

Parametros: Nenhum

Payload / Request Body (JSON):

```
{
  "serviceId": "service-1",
  "providerId": "provider-1",
  "clientName": "Marcos Andrade",
  "clientEmail": "marcos@email.com",
  "clientPhone": "(11) 98888-7777",
  "startTime": "2026-08-20T14:00:00.000Z"
}
```

Exemplo de Resposta (Status 200/201):

```
{
  "id": "bk-1723...",
  "status": "confirmed",
  "serviceName": "Corte de Cabelo Classico",
  "startTime": "2026-08-20T14:00:00.000Z",
  "endTime": "2026-08-20T14:30:00.000Z"
}
```

PATCH **/api/bookings/:id****[AUTH]** Requer JWT

Atualiza o status de um agendamento (confirmed, completed ou cancelled). Requer token JWT.

Parametros: :id (String - obrigatorio)

Payload / Request Body (JSON):

```
{
  "status": "completed"
}
```

Exemplo de Resposta (Status 200/201):

```
{
  "id": "bk-1",
  "status": "completed"
}
```

DELETE **/api/bookings/:id****PUBLICO**

Exclui permanentemente um agendamento do banco de dados.

Parametros: :id (String - obrigatorio)

Exemplo de Resposta (Status 200/201):

```
{ "success": true }
```

GET **/api/bookings/my****[AUTH]** Requer JWT

Retorna o historico completo de agendamentos pertencentes ao cliente autenticado via JWT.

Parametros: Nenhum (identificado pelo token)

Exemplo de Resposta (Status 200/201):

```
[
  {
    "id": "bk-1",
    "serviceName": "Corte Classico",
    "providerName": "Carlos Barber",
    "startTime": "2026-08-20T14:00:00.000Z",
    "status": "confirmed"
  }
]
```

PATCH **/api/bookings/:id/reschedule****[AUTH]** Requer JWT

Permite ao cliente autenticado reagendar o horario de sua reserva com verificacao de disponibilidade em tempo real.

Parametros: :id (String - obrigatorio)

Payload / Request Body (JSON):

```
{
  "newStartTime": "2026-08-22T15:00:00.000Z"
}
```

Exemplo de Resposta (Status 200/201):

```
{
  "id": "bk-1",
  "startTime": "2026-08-22T15:00:00.000Z",
  "status": "confirmed"
}
```

8. Modulo de Autenticacao e Usuarios

Registro, login seguro com bcrypt e sessao JWT

POST /api/auth/register

Publico

Cria uma nova conta no sistema (Cliente ou Profissional). Se for profissional, cadastra os dados adicionais de biografia e especialidade.

Parametros: Nenhum

Payload / Request Body (JSON):

```
{
  "name": "Juliana Costa",
  "email": "juliana@estetica.com",
  "password": "senhaSegura123",
  "role": "provider",
  "tenantId": "tenant-2",
  "categoryId": "cat-2",
  "bio": "Esteticista com 8 anos de experiencia."
}
```

Exemplo de Resposta (Status 200/201):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "user-1723...",
    "name": "Juliana Costa",
    "email": "juliana@estetica.com",
    "role": "provider",
    "providerId": "provider-1723..."
  }
}
```

POST /api/auth/login

Publico

Autentica o usuario comparando a senha com o hash bcrypt e retorna o token JWT e o perfil associado.

Parametros: Nenhum

Payload / Request Body (JSON):

```
{
  "email": "carlos@imperial.com",
  "password": "12345678"
}
```

Exemplo de Resposta (Status 200/201):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "user-1",
    "name": "Carlos Barber",
    "email": "carlos@imperial.com",
    "role": "provider",
    "providerId": "provider-1"
  }
}
```

POST /api/auth/social-login

Publico

Efetua login ou registro social simplificado (Google / Instagram).

Parametros: Nenhum

Payload / Request Body (JSON):

```
{
  "email": "usuario@gmail.com",
  "name": "Roberto Santos",
  "provider": "google"
}
```

Exemplo de Resposta (Status 200/201):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "user-social-...",
    "email": "usuario@gmail.com",
    "name": "Roberto Santos",
    "role": "client"
  }
}
```

GET

/api/auth/me**[AUTH]** Requer JWT

Obtem os dados do usuario a partir do token JWT enviado no cabecalho Authorization.

Parametros: Nenhum (usa cabecalho Authorization)

Exemplo de Resposta (Status 200/201):

```
{
  "user": {
    "userId": "user-1",
    "email": "carlos@imperial.com",
    "name": "Carlos Barber",
    "role": "provider",
    "providerId": "provider-1"
  }
}
```

9. Modelos de Dados e Esquema TypeScript

Definicoes das entidades principais do banco de dados

```
// 1. Tenant (Estabelecimento)
interface Tenant {
  id: string;
  name: string;
  slug: string;
  logoUrl: string;
  description: string;
  themeColor: 'indigo' | 'emerald' | 'rose' | 'amber';
  accentColor: string;
}

// 2. Provider (Profissional)
interface Provider {
  id: string;
  tenantId: string;
  categoryId?: string;
  name: string;
  email: string;
  bio?: string;
}

// 3. Service (Servico)
interface Service {
  id: string;
  providerId: string;
  name: string;
  description: string;
  durationMinutes: number;
  bufferMinutes: number;
  price: number;
}

// 4. Booking (Agendamento)
interface Booking {
  id: string;
  serviceId: string;
  providerId: string;
  clientName: string;
  clientEmail: string;
  clientPhone: string;
  startTime: string; // ISO 8601 UTC
  endTime: string; // ISO 8601 UTC
  status: 'confirmed' | 'cancelled' | 'completed';
  price?: number;
}
```